



Security Assignment — Repository VAPT Tool

Build a tool that runs from inside a repository and does a VAPT of it end to end — the code, the running application, and its APIs. We drop it into any of our Python projects, run it, and get one report telling us what is wrong and what to fix first.

CANDIDATE	Lekhranj Singh
TIME	7–10 days
DUE	_____

How it runs

From the root of the repo being tested, one command:

- `python -m repoguard scan .` — static assessment only.
- `python -m repoguard scan . --url http://localhost:8000` — also scans the running app and its APIs.
- `--token` to supply a valid auth token when the app has one, so protected endpoints can be tested too.
- The app is started by whoever runs the tool. You do not have to figure out how to boot someone else's project.
- Ships with a Dockerfile or compose file so all the scanners come with it — no manual tool installation.
- Exits with a non-zero code if there are critical findings.

Part 1 — Static assessment

- **Dependencies** — known CVEs in `requirements*.txt`, `poetry.lock` or `Pipfile.lock`, with severity and the fixed version. Use `pip-audit`.
- **Code** — run `bandit` and `semgrep` over the project: injection, command execution, unsafe deserialization, weak crypto, path traversal.
- **Secrets** — keys, tokens and credentials in the code and in git history. Use `gitLeaks`.
- **Configuration** — **write this part yourself**. Read the project's Django or Flask settings and flag: `DEBUG=True`, `ALLOWED_HOSTS=['*']`, a hardcoded `SECRET_KEY` or database password, missing CSRF and security middleware, session and CSRF cookies without the secure flag, `CORS_ALLOW_ALL_ORIGINS=True`, and `app.run(debug=True)`.
- **Repo hygiene** — `.env` files, private keys, certificates or database dumps committed to the repository.

Part 2 — API discovery

Read the project's routing and build a list of every endpoint it exposes — this is what the API testing runs against, and it is useful to us on its own:

- Parse Django `urls.py` and DRF routers, or Flask `@app.route` decorators, to get every path, its HTTP methods, and the view behind it.
- For each endpoint, record what protects it — the authentication and permission classes on the view, or `@login_required`, or nothing at all.
- Flag every endpoint with no authentication or permission applied, and every one left on the project's global default when that default is `AllowAny`.
- Include the endpoint inventory in the report as a table.

Part 3 — Live application and API testing

When a URL is given, test the running application and put those findings in the same report.

Application

- Missing or weak security headers — CSP, HSTS, X-Frame-Options, X-Content-Type-Options.
- Cookies without `Secure`, `HttpOnly` or `SameSite`.
- Exposed paths — `/admin`, `/.env`, `/.git`, debug pages, directory listing, server and framework version disclosure.
- Stack traces or the Django debug page returned on an error.

APIs — take the endpoints found in Part 2 and actually call them:

- Call every endpoint **with no token**. Anything that returns data instead of 401 or 403 is a critical finding — this is the single most common real API bug.
- Call each one with every HTTP method it did not declare. A `DELETE` or `PUT` that works where only `GET` was intended is a finding.
- Send a malformed or oversized body and check the error response does not leak a stack trace, a SQL query, or internal paths.
- Check whether repeated requests are rate limited at all.
- **Object-level access** `bonus` — given two users' tokens, use one user's token to request the other user's object IDs. If it succeeds, that is an IDOR.

The report

- One HTML report plus a `findings.json`, written into an output folder in the repo.
- Every finding: severity, where it is (file and line, or the endpoint and method), the offending snippet or the actual request and response, the OWASP category, and a specific fix.
- A summary at the top — counts by severity and the issues to fix first.
- The endpoint inventory, showing which endpoints are protected and which are not.
- A report with 400 findings is worth less than one that surfaces the 12 that matter. Sort by what actually needs fixing.

Prove it works

- Run the full scan against [PyGoat](#) — an intentionally vulnerable Django app that starts with its own compose file, so you can do the static and the live scan on it.
- For the API part, run it against a small Django REST Framework project — a public sample, or a throwaway one you write with two or three endpoints, one of them deliberately left unprotected.
- Commit both reports.

Submit

- A **public** Git repository with a README: how to run it, what you decided and why, and what is not done.
- The generated reports.
- A short screen recording of a live run.

Rule

It must run end to end on a clean machine by following your README. A smaller tool that works beats a bigger one that half-works.